

Projeto Multidisciplinar - Trilha Back-End

Rede "Raízes do Nordeste"

Aluno: [Wesley Guimarães Marinho / RU 4736850]

Polo de Apoio: CENTRO UNIVERSITÁRIO INTERNACIONAL UNINTER

PROJETO MULTIDISCIPLINAR DE ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

TRILHA: BACK-END

Professor: Prof. Me. Luciane Yanase Kanashiro

Semestre/Ano: 2026

Sumário

1. Introdução
 2. Análise e Requisitos
 3. Modelagem e Arquitetura
 4. API e Endpoints
 5. LGPD, Privacidade e Segurança
 6. Entrega Técnica
 7. Plano de Testes
 8. Conclusão
 9. Referências
-

1. Introdução

Este trabalho apresenta o desenvolvimento da API (Back-End) para o estudo de caso da rede de lanchonetes "Raízes do Nordeste". Como o negócio está crescendo bastante e possui diferentes unidades, o foco do projeto foi criar um sistema capaz de receber pedidos de vários canais diferentes (App, Totem, Balcão, Web e Retirada) sem perder o controle e a rastreabilidade.

A ideia foi construir uma arquitetura funcional em Python, permitindo a gestão dos pedidos, o controle do cardápio e simulando a integração com um gateway de pagamentos externo (Mock). Tudo isso pensando em manter o código organizado e aplicar na prática os conceitos vistos durante o curso.

2. Análise e Requisitos

Requisitos Funcionais (RF)

- **RF01:** Cadastro e Autenticação de usuários, diferenciando os perfis em ADMIN, GERENTE e CLIENTE.
- **RF02:** Visualização de cardápio, permitindo a listagem e consulta detalhada de produtos.
- **RF03:** Realização de pedidos multicanal. A API deve exigir e registrar o canalPedido (APP, TOTEM, BALCAO, WEB, PICKUP).
- **RF04:** Atualização de status de pedido (CRIADO -> PAGO -> COZINHA -> ENTREGUE / CANCELADO).
- **RF05:** Mock de pagamento. O sistema deve validar a forma de pagamento externa, alterando o status caso seja "MOCK" ou simulando recusa.
- **RF06:** Controle de estoque com restrição de vendas, decrementando saldo ou bloqueando caso não haja produto.
- **RF07:** Fidelização (base: acumular pontos).

Requisitos Não Funcionais (RNF)

- **RNF01:** Segurança da informação. Uso de senhas criptografadas (Hash BCrypt) e JWT (JSON Web Tokens) para as chamadas de API.
- **RNF02:** Aderência à LGPD (coleta mínima de dados, finalidade justificada, hash de senhas e anonimização quando aplicável).
- **RNF03:** Arquitetura limpa/modular (divisão em Routers, Schemas, Models, Services).
- **RNF04:** API totalmente documentada via Swagger/OpenAPI.
- **RNF05:** Padrão uniforme de erro JSON para todos os endpoints.

3. Modelagem e Arquitetura

3.1. Diagrama de Casos de Uso (Conceitual)

Atores Identificados:

1. **Cliente (App/Web/Totem):** Interage com o sistema para consumir serviços.
2. **Atendente (Balcão):** Funciona como a operação da loja física.

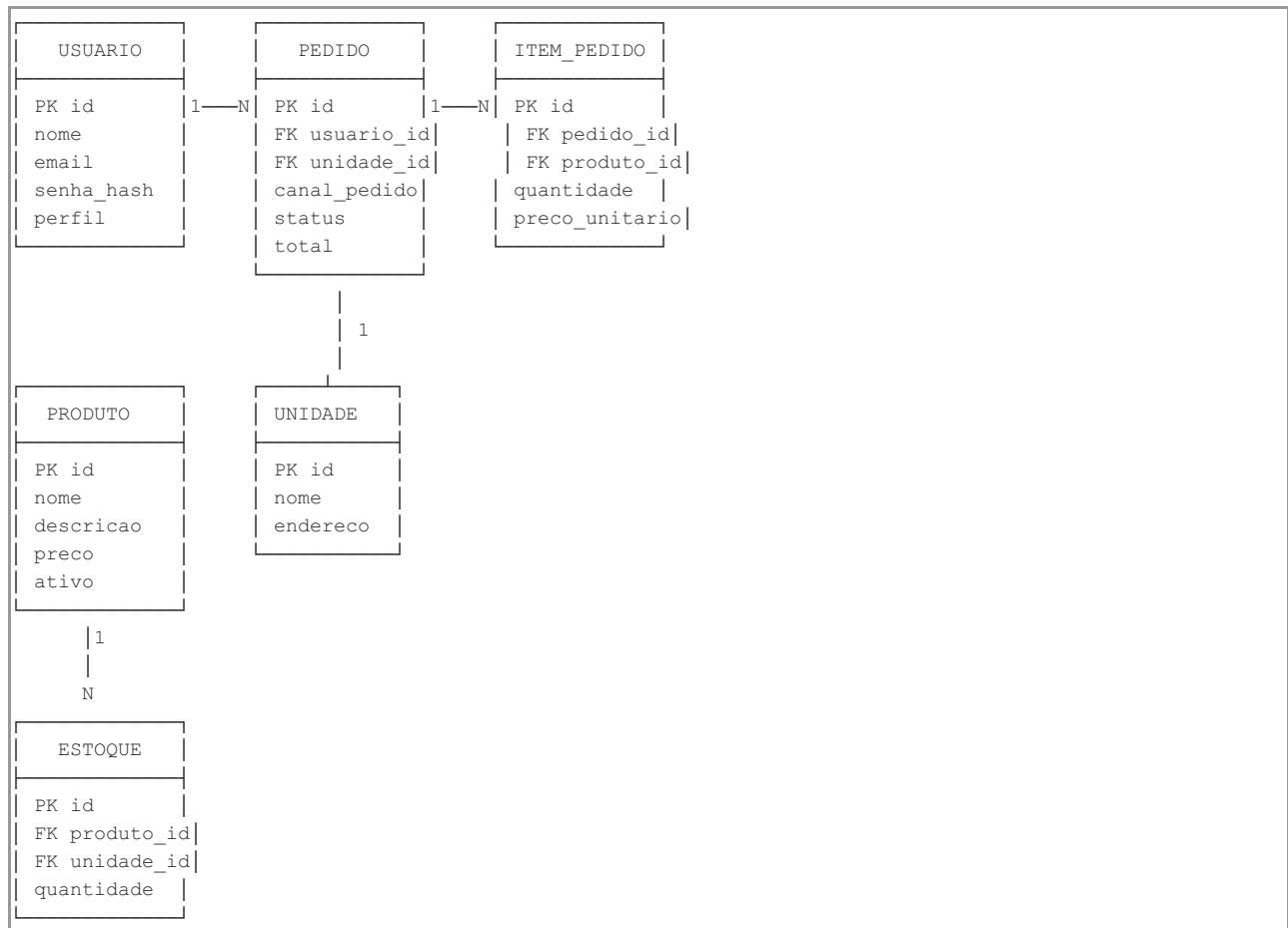
3. **Cozinha:** Recebe os pedidos já pagos para preparar.
4. **Gerente / Administrador:** Controla estoque e produtos.
5. **Gateway de Pagamento:** Sistema externo de processamento.

Casos de Uso Obrigatórios:

1. **UC01 - Realizar Pedido (Ator: Cliente / Atendente):** O ator seleciona os itens e o canal de pedido. O sistema valida as informações e insere o registro com status CRIADO.
2. **UC02 - Gerenciar Estoque (Ator: Gerente):** O gerente insere ou retira o saldo de produtos atrelados a uma unidade.
3. **UC03 - Processar Pagamento Mock (Ator: Gateway):** Confirmação de recebimento. Atualiza status para PAGO se sucesso ou recusa com mensagem padrão.
4. **UC04 - Gerenciar Fidelidade (Ator: Cliente):** Consulta de pontos adquiridos e resgate de recompensas.

3.2. Modelo de Dados (DER) e Diagrama de Classes

Abaixo segue o Diagrama Entidade-Relacionamento (DER) representando as tabelas do banco de dados (SQLite via SQLAlchemy):



3.3. Arquitetura em Camadas

O projeto utiliza a linguagem Python com o framework FastAPI, estruturando-se em:

- **API/Routers:** Camada de transporte (recebe requests, responde JSON).
- **Schemas:** Definição de contratos (Pydantic models) e validações.
- **Models/Infrastructure:** Definição das tabelas do banco (SQLAlchemy) e conexão.
- **Services/Domínio:** Onde apliquei as regras de negócio (cálculo de total, simulação do mock de pagamento, etc).

4. API e Endpoints

Abaixo estão os endpoints documentados conforme exigido no roteiro. A documentação interativa também está no Swagger (</docs>).

Padrão de Erro do Sistema:

```
{
  "error": "VALIDATION_ERROR",
  "message": "Mensagem detalhada do erro.",
  "timestamp": "2026-06-23T17:00:00Z",
  "path": "/api/exemplo"
}
```

Checklist de Endpoints

1. Login (POST /auth/login)

- *Propósito:* Autenticar usuário e retomar JWT.
- *Permissões:* Público.
- *Parâmetros:* Body JSON {"email": "...", "senha": "..."}.
- *Response (200):* {"access_token": "...", "token_type": "bearer"}

2. Listar Cardápio da Unidade (GET /unidades/{id}/cardapio)

- *Propósito:* Listar os produtos ativos de uma unidade.
- *Permissões:* Público.
- *Parâmetros:* Path param id (int).
- *Response (200):* [{"id": 1, "nome": "Tapioca", "preco": 10.0}]

3. Criar Pedido (POST /pedidos/)

- *Propósito:* Registrar pedido com itens e canal.
- *Permissões:* Requer JWT (Bearer).
- *Parâmetros:* Body JSON {"canalPedido": "APP", "itens": [{"produto_id": 1, "quantidade": 2}]}
- *Response (201):* JSON do Pedido criado contendo os detalhes inseridos e total.

4. Atualizar Status do Pedido (PATCH /pedidos/{id}/status)

- *Propósito:* Atualizar o status do pedido (ex: de PAGO para COZINHA).
- *Permissões:* Requer JWT (Bearer).
- *Parâmetros:* Path id. Body JSON {"forma_pagamento": "..."}.
- *Response (200):* {"detail": "Status atualizado.", "novo_status": "COZINHA"}

5. Processar Pagamento Mock (POST /pagamentos/)

- *Propósito:* Simular o gateway de pagamento externo e alterar status para PAGO.
- *Permissões:* Requer JWT (Bearer).
- *Parâmetros:* Body JSON {"pedido_id": 1, "forma_pagamento": "MOCK"}
- *Response (200):* {"detail": "Pagamento simulado com sucesso via Gateway MOCK.", "novo_status": "PAGO"}

6. Consultar Saldo Fidelidade (GET /fidelidade/saldo)

- *Propósito:* Checar quantos pontos o usuário possui.
- *Permissões:* Requer JWT.
- *Parâmetros:* Nenhum (usa token).
- *Response (200):* {"usuario_id": 1, "saldo_pontos": 150}

7. Registrar Pontos Fidelidade (POST /fidelidade/regarstrar)

- *Propósito:* Acumular pontos na conta do cliente ao efetuar compras.
- *Permissões:* Requer JWT.
- *Parâmetros:* Query param pontos.
- *Response (200):* {"detail": "Pontos registrados com sucesso.", "saldo_total": 200}

8. Resgatar Pontos Fidelidade (POST /fidelidade/resgatar)

- *Propósito:* Trocar pontos por benefícios.
- *Permissões:* Requer JWT.
- *Parâmetros:* Query param pontos.
- *Response (200):* {"detail": "Pontos resgatados...", "saldo_restante": 50}

9. Consultar Estoque Unidade (GET /estoque/{unidadeId})

- *Propósito:* Visualizar saldos de produtos da unidade.
- *Permissões:* Público (ou Logado).
- *Parâmetros:* Path param unidadeId.
- *Response (200):* [{"produto_id": 1, "quantidade": 50}]

10. Movimentar Estoque (PATCH /estoque/movimentar)

- **Propósito:** Realizar entrada/saída de itens.
- **Permissões:** Apenas GERENTE/ADMIN (validação no JWT).
- **Parâmetros:** Body JSON {"produto_id": 1, "unidade_id": 1, "quantidade": 10}
- **Response (200):** {"detail": "Movimentação registrada com sucesso.", "novo_saldo": 60}

5. LGPD, Privacidade e Segurança no Back-End

Para atender aos requisitos de segurança e LGPD propostos pelo estudo de caso, procurei manter o armazenamento apenas do que é essencial para o fluxo funcionar.

- As senhas dos usuários nunca são salvas abertamente (usei a biblioteca nativa `bcrypt` para salvar apenas o hash no banco).
- Apliquei um esquema de autenticação com tokens JWT.
- Para fins de auditoria, a API gera logs de sistema sempre que uma ação sensível ocorre, como a criação, movimentação de estoque ou a alteração de status de um pedido. Isso garante total rastreabilidade sobre as movimentações e seus autores ("quem fez e quando").
- Rotas de controle sensível exigem que o usuário tenha o perfil de administrador ou gerente.
- Quando um cliente loga, ele consegue listar e visualizar apenas os seus próprios pedidos, evitando expor dados de outras pessoas e garantindo a privacidade.

6. Entrega Técnica

- **Código-Fonte e Repositório:** O código encontra-se implementado e pode ser verificado no repositório público do GitHub do aluno.
- **Swagger Local:** Ao rodar a API, acessível via `http://localhost:8000/docs`.
- **Coleção Postman:** Disponibilizada como arquivo no diretório raiz do projeto.
- **README:** Contém as instruções de setup e testes.

7. Plano de Testes

Os testes da aplicação seguem a estrutura abaixo e podem ser executados através da coleção do Postman importada.

ID	Endpoint	Pré-condição	Entrada (Resumo)	Saída Esperada	Evidência (Pasta/Nome)
T01	POST /auth/login	Usuário existe na base	JSON {email, senha} válido	200 + Access Token no body	Auth / Login válido
T02	GET /pedidos/	Usuário deslogado	Não informar Token Header	401 Unauthorized + JSON Erro	Auth / Acesso Sem Token
T03	POST /pedidos/	Usuário logado	Sem campo canalPedido	422 Unprocessable + JSON Erro	Pedidos / Falta Canal
T04	POST /pedidos/	Usuário logado, Produto 1 ativo	canalPedido=APP e item id=1	201 Created + JSON do Pedido	Pedidos / Criar Pedido
T05	POST /pedidos/	Usuário logado	Produto = 999 (Inexistente)	404 Not Found + Erro padrão	Pedidos / Produto Inexistente
T06	POST /pagamentos/	Pedido 1 como CRIADO	{pedido_id: 1, forma_pagamento: "MOCK"}	200 + Status alterado p/ PAGO	Pagamentos / Pagar MOCK Ok
T07	POST /pagamentos/	Pedido 2 como CRIADO	{pedido_id: 2, forma_pagamento: "FALHA"}	400 Bad Request + Pag. Recusado	Pagamentos / Pagar MOCK Recusado
T08	PATCH /estoque/movimentar	Logado c/ perfil CLIENTE	JSON Movimentação de estoque	403 Forbidden (Sem permissão)	Estoque / Acesso Negado
T09	GET /fidelidade/saldo	Logado c/ qualquer perfil	Endpoint acionado com Token	200 + Pontos retornados (150)	Fidelidade / Consultar Pontos
T10	POST /fidelidade/resgatar	Logado, possui 150 pts	Query param pontos=200	400 Bad Request + Insuficiente	Fidelidade / Saldo Insuficiente

8. Conclusão

Desenvolver este projeto foi um ótimo desafio para colocar em prática o que foi ensinado na disciplina. Consegui montar uma API com FastAPI que não apenas atende aos requisitos do estudo de caso, como o uso do `canalPedido` e a simulação de pagamento via gateway (Mock), mas que também funciona de verdade integrado a um banco de dados relacional.

Optei por usar o SQLite para deixar o projeto mais fácil de ser executado e testado por quem for corrigir, sem perder a complexidade da estrutura de tabelas (incluindo FKs de Unidade, etc) usando o SQLAlchemy. Acredito que o código ficou organizado, bem documentado através do Swagger automático e que essa solução se saíra muito bem em uma demonstração para vagas na área.

9. Referências

- Documentação do FastAPI. Disponível em: <https://fastapi.tiangolo.com/>

- Material de Eletiva IV - Projeto Multidisciplinar - UNINTER.